

Reproducible Science

Machine-checkable claims from preregistration to publication

Elliot Tower

NeurIPS 2026 Workshop on AI for Science

Overview

Part 1: The problem

1. A paper is a pile of claims
2. Re-running is not verifying
3. Verification debt
4. Three boundaries
5. Nothing failed because nothing ran

Part 2: How it works

6. Four commands
7. Evidence assertions
8. Three stages, five outcomes
9. Facts, then policy
10. Checked while you work
11. Files it reads
12. Session status
13. A record out of step
14. Outside timestamps

Part 3: What it found

15. The paper checks itself
16. Six PDF readers, six answers
17. 113 failures, zero misquotations
18. Recall on CORE-Bench
19. Six defect classes
20. What a reader can re-check from a clone
21. An edit to a registered plan
22. Use across a corpus

Part 4: In context

23. Related work
24. What it does not establish
25. Summary
26. Try it

Part 1

The problem

A paper is a pile of claims

Every sentence that states a number or quotes a source is a claim that something is true **somewhere else**:

“accuracy was 0.42”

points at a results file

“Smith writes that. . .”

points at a PDF

“a confirmatory test”

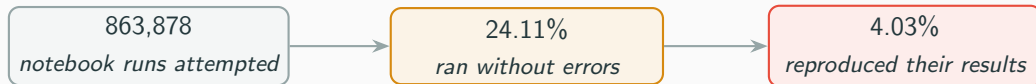
points at a plan

AI tools now generate hypotheses, run analyses and draft manuscripts quickly. Checking that the finished paper still agrees with what is behind it has not sped up with them.

Today that check is done by hand or by asking a model. Both are expensive, hard to audit, rarely say how much they checked, and go stale as soon as the project changes.

Re-running is not verifying

The usual answer is to re-run the analysis. It is usually unavailable:



Pimentel et al. (2019), valid Jupyter notebooks on GitHub.

And even a perfect re-run checks the wrong thing. It shows the **code runs**. It says nothing about whether the **paper** prints what the code produced.

A project can keep its code, data and logs and still print a value no output contains, quote a passage no reader found, or call an analysis confirmatory after its plan changed.

Verification debt

Like technical debt in software, it is cheap to catch when it happens and expensive to reconstruct later.

When a value changes

The run that produced the old value is still on disk. The source still resolves. The edit has an author and a commit.

Months later

The run was overwritten. The PDF was fetched once and its address not kept. Which document a citation meant is no longer recoverable.

What you need to settle a disagreement is present when the disagreement appears, and decays afterwards. So the check has to run **while the work happens**, not once before submission.

Three boundaries

A research project has three seams where a claim can come loose from its evidence:

Boundary	What goes wrong unchecked	What gets compared
Plan to run	The hypothesis or analysis is rewritten after the result is visible	A run record against a plan pinned by hash
Run to manuscript	A printed number drifts from the output it came from	A declared address in a sealed results file
Source to manuscript	A quotation does not occur in the source cited for it	The passage against a hash-pinned source

Most of what a reader wants checked is not mechanical: whether the design answers the question, whether the conclusion follows. **This narrow part is:** a number either matches its file or it does not, and a quote either occurs in the source or it does not.

Nothing failed because nothing ran

Two incidents from building the tools themselves:

A reader that read nothing

A PDF extractor was missing, so it returned an empty string. The check recorded an unreadable document, and the summary said *nothing failed*.

A check that checked no files

A folder was renamed, so a script's file pattern matched nothing. It exited successfully. CI, a pre-push hook and a make target were all green.

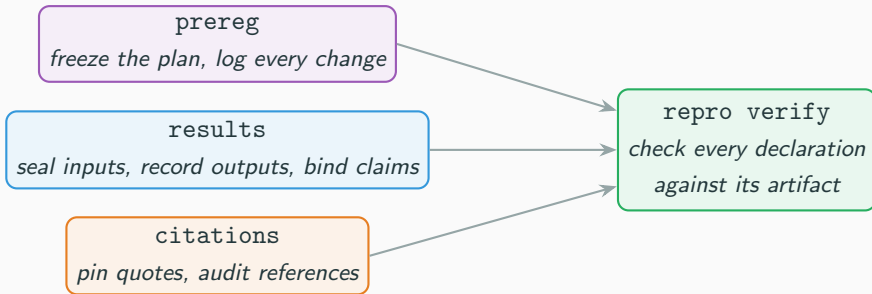
A green result is not evidence that anything was examined. A report needs a separate word for **“could not check”**, or it cannot tell you that a passing build looked at nothing.

Part 2

How it works

Four commands

Three commands write declarations as the work happens. The fourth reads them all.



- prereg: the plan that ran is the plan that was frozen, against a commit and a hash.
- results: a reported number came from a recorded run over sealed inputs.
- citations: a quoted passage occurs in the source cited for it.

Evidence assertions

The unit is one printed value bound to one **declared address** in a hash-pinned file.
Four kinds are implemented:

Kind	Assertion	Located by	Compared how
metric	this file holds this value here	a JSON pointer	at the precision printed
table	this table holds this value in this cell	a column and a row selector	at the precision printed
quote	this passage occurs in this source	a page, optionally	word for word, after normalization
correspondence	two files hold the same value	a locator on each side	at the precision printed

Searching afterwards for a number that looks similar is not provenance. A table cell is found by its key columns, not its row number, so reordering cannot point at a different cell.

Three stages, five outcomes

Every check has to **run**, **extract** and **compare**, and each can fail on its own. The report keeps them apart:

Situation	Run	Extract	Compare	Outcome
value present	completed	extracted	match	verified
value differs	completed	extracted	mismatch	mismatch
address does not resolve	completed	absent	—	not_found
extractor not installed	unavailable	not tried	—	unchecked
bug in the checker	failed	not tried	—	error

Each row asks you to do something different: fix the claim, fix the address or run the analysis, install a package, or file a bug. A pass/fail report can't tell you which.

Facts, then policy

The checker reports facts and computes no verdict. A **profile** decides what counts as a failure:

Profile	mismatch	not_found	unchecked	not offered	unpinned
exploratory	error	warning	warning	ignore	ignore
publication	error	error	warning	warning	warning
confirmatory claims	error	error	error	error	—
strict	error	error	error	error	error

An unchecked citation is fine in a draft and disqualifying in a submission. Re-grading under a stricter profile re-reads the stored report and **runs nothing**, so a submission is judged on the facts the draft was checked with.

Every profile requires at least one check to have run, or a project with no evidence would pass everything.

Checked while you work

- **Ordinary files.** Any language, notebook, framework or agent; the manuscript need not be written inside the toolchain.
- **Declare as you go.** Freeze the plan before the run, seal inputs before the analysis, record outputs after, and name where a number came from when you write it down.
- **Agent plugin.** A Claude Code plugin records results during the experiment and reports drift on every edit, so declarations are written when artifacts are made.
- **Preregistration is optional, per claim.** Only claims marked confirmatory need a plan, and they are held to the plan they name.
- **Plan edits are visible, never blocked.** The hook fires on every edit and refuses none, because a check that can halt the work gets switched off.

Files it reads

The checker runs on the files a project already has. Nothing is converted and no manuscript is rewritten to be auditable.

Evidence	Read from
A value	JSON, delimited text, a workbook, Parquet, Stata, SPSS, R data, HDF5, NetCDF
A quotation	a PDF, plain text, a workbook, a Word document, JATS XML

A format with no reader installed returns **unchecked**, never a pass and never a mismatch.

The fifteen papers in the quotation audit were written without this contract in mind, and none was modified to be auditable.

Session status

A Claude Code mod draws the project's records above the prompt after each turn, one row per command.

The project audited later in this talk: every plan frozen, 131 claims bound to 13 runs.

A literature survey in progress: 7,783 quotations pinned, no claim yet bound.

A record out of step

A second line appears under the prompt only when a record disagrees with what was registered.

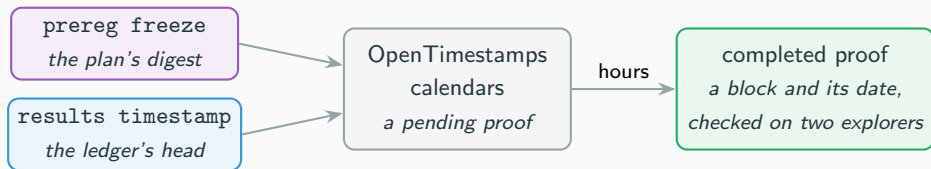
```
warning-demo · prereg: 1 changed [-]  
results: 1 run, 0 sealed, 0 claims  
citations: none pinned  
  
----- warning demo -----  
>   
  
△ repro: 1 plan edited after freeze · 1 run, nothing sealed
```

Here a frozen plan was edited and a run was recorded with no input sealed. The line names both.

The other warnings: **sealed file changed, ledger rewritten after timestamp, runs recorded, no plan frozen.**

Outside timestamps

A freeze is a digest stored in the repository, and whoever holds the repository can rewrite both. A timestamp held outside it fixes the latest date the plan could have been written.



- **No account and no fee.** The digest is salted before it is sent, and at least two calendars must accept it.
- **A rewrite shows.** `results verify` reports a chain rewritten after it was stamped.
- **Optional.** `--no-timestamp` freezes and sends nothing.

Part 3

What it found

The paper checks itself

The paper is an instance of what it describes. **17 claims**, bound by **34 assertions** to **7 artifacts**, pass under the strictest profile. A reader re-runs the check; they don't take it on trust.

Building it found two errors in the paper's own numbers:

Computed, written nowhere

Four totals the paper quoted had been computed correctly but saved in no file, so nothing could re-check them.

Stale, twice

A stored count said the test suite had 19 fixtures; it has 25. An earlier audit had found it stale once already.

Limit: the paper, the manifest and the checker share an author, so this is an integration demonstration, not independent validation.

Six PDF readers, six answers

Extractor	resolved	seconds
poppler, reading order	2,314	6.9
poppler, layout	2,250	6.9
pure-Python reader	2,229	43.9
rendering-engine binding	2,092	4.8
structural extractor	1,973	6.2
low-level parser	1,716	2.7
both poppler modes	2,360	13.8
all six	2,362	71.4

Passages resolved, of 2,364, from the same pinned bytes.

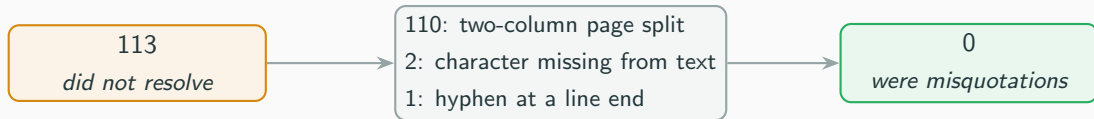
Six extractors read the same 2,364 quotations out of the same files and returned six different answers.

So a verification snapshot records the **reader** as much as the corpus. A rerun with a different toolchain may not find the same issues.

The weakest reader is the only one that adds passages beyond the two poppler modes. Two quotations resolve under no reader at all.

113 failures, zero misquotations

An audit of **2,512 quotations** against **125 hash-pinned sources**:



Every failure came from **extraction**, not from the author. A pass/fail checker would have recorded all 113 as misquotations:

- a **4.5%** false-accusation rate against the manuscript, and
- a misquotation-detection rate of **zero**.

Adjudicated by one rater. The pins guarantee the file checked is byte-identical to the one cited.

Recall on CORE-Bench

CORE-Bench capsules state which values their analysis produces, so recall can be measured against a known answer set.

Strength	stated	found	recall
trivial	59	54	92%
weak	76	40	53%
moderate	44	15	34%
strong	61	37	61%
all	240	146	61%

45 capsules. Strength = number of constraining digits.

Recall falls as values carry more digits, then rises again. The tiers differ in what they are made of: a two-digit integer turns up in almost any file, and a long decimal only where it was written.

Eight capsules hold a file no reader here could open, so their misses are reported as **unchecked**, not **absent**. A pass/fail report would count all of them against the capsule.

Six defect classes found in ordinary use

Class	What is observably wrong	Status
stored value $\neq$ printed value	9 of 10 table rows disagree with their file	open
false mismatch under float equality	3 of 39 agreements hold only under decimal comparison	closed
hash recorded, never compared	digests recorded and never checked against the files	closed
check that runs on no files	a pattern matches nothing, the check exits zero, the build is green	open
claims split from their evidence	178 assertions marked verified, no source in any commit	closed locally
figure that stopped reproducing	a count no longer reproduces from its bibliography	open

In the first row every file matches its recorded hash, so no integrity check fires. A value printed as 0.001 is stored as exactly zero. Only a check that compares the file to the sentence citing it can see that.

What a reader can re-check from a clone

The four commands over a clone of a public repository developed with the toolkit, at a pinned commit.

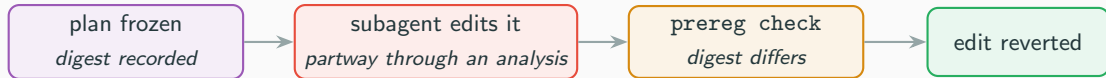
Command	Found	Could not establish
prereg	3 plans, each unchanged against the digest recorded at its freeze	whether a plan was written before its outcomes were seen
results	chain intact at 153 events; 8 of 12 recorded files match their digest, 4 carry more than one	what share of the manuscript's numbers the 131 claims cover
citations	18 quotations against 7 sources, each with a SHA-256; none checked	whether any quotation occurs in its source
repro	no evidence manifest; exit 2	any assertion, none being declared

A publisher's PDF is not the author's to redistribute, so the repository carries each source's digest.

`citations fetch` downloads the pinned sources and keeps only bytes that match the pin.

An edit to a registered plan

A frozen plan carries a recorded digest. An edit is found by recomputing the digest, not by asking the author what changed.



This happened in ordinary use. The agent doing the analysis was the one that changed the plan.

Amending a registration is legitimate, and prereg log records it. Amending it silently is what the digest makes impossible.

Use across a corpus

The commands have run during real work, not over a corpus built to be audited:

22 repositories

466 claim files

5,590

declared quotations

23 frozen plans

all still verify

A sweep of 25 repositories found 21 where at least one command applied, and 4 declaring nothing, which the report states rather than counting as a pass.

Fast enough for every commit. Verifying 12 to 34 assertions over 3 to 7 files takes 0.83 to 0.91 seconds. Resolving 61 quotations against 12 sources takes 3.69 seconds.

One author's use: it shows the declarations hold up in ordinary work, and nothing about other people's conventions.

Part 4

In context

Related work

System	What it does	How this differs
Paperclip	binds a claim to a line range in a source, checked by a model	a verdict you can recompute; addresses into pinned bytes, not a re-parse
adduce	scores a repository against 78 rules	one value, one address, no score; the two compose
Code Ocean, Whole Tale	verify that a capsule runs	checks which of the paper's values a file still supports
FEVER, SciFact	predict whether text supports a claim	reports what a byte or decimal comparison found
Literate programming	generates the paper from the code	reports drift instead of absorbing it; usable together

adduce asks the same question independently: *for every number in the paper, can I point to the artifact that produced it?*

What it does not establish

- **The extractor is trusted.** A hash shows the file did not change, not that it was read correctly. A parser that reads 0.9489 as 0.9439 leaves the hash intact. Recording the reader makes a changed reading visible, which catches drift, not error.
- **Registration shows order, not honesty.** It shows the plan was registered before the run began. An outside timestamp fixes the latest date a plan could have been written. It cannot show the plan was not written earlier and withheld.
- **Coverage can be gamed.** Someone writes every assertion, so a project that declares 5% of its numbers is checked on 5%, and choosing which 5% is a way to pass. The report states coverage beside outcomes, so it is visible, not prevented.

Summary

Re-running an analysis
shows the code runs

Reproducible Science
checks what the paper says it produced

- Numbers, quotations and registered plans differ only in how they are extracted, so one engine checks all three.
- Disagreement, absence and “could not check” need different fixes, so the report keeps them apart.
- The check is cheap enough to run on every commit, which is when the evidence to settle a disagreement still exists.

```
pip install reproducible-science
```

- Documentation and an in-browser demo: elliotttower.github.io/reproducible-science
- Code and the Claude Code plugin: github.com/elliotttower/reproducible-science
- The four tools install separately too: `prereg`, `citations`, `results-cli`, `reproducible-science`.

Reproducible Science: Machine-Checkable Claims from Preregistration to Publication. Elliot Tower. NeurIPS 2026 Workshop on AI for Science.